Operating Systems Week 3 Lecture 3

Junaid Abdul Wahid

Previous

- Processes
- Process Control Block

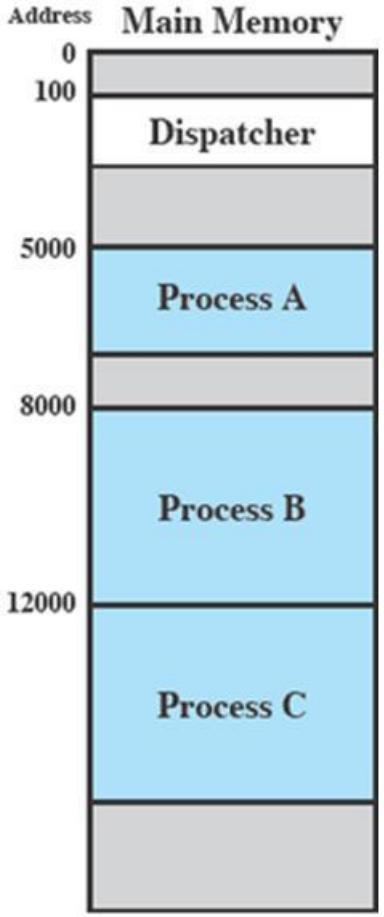
Today Agenda

- Trace of the Processes
- Process Execution
- Operating System Control Structures

Trace of the Process

The behavior of an individual process is shown by listing the sequence of instructions that are executed
This list is called a ***Trace***
Dispatcher is a small program which switches the processor from one process to another

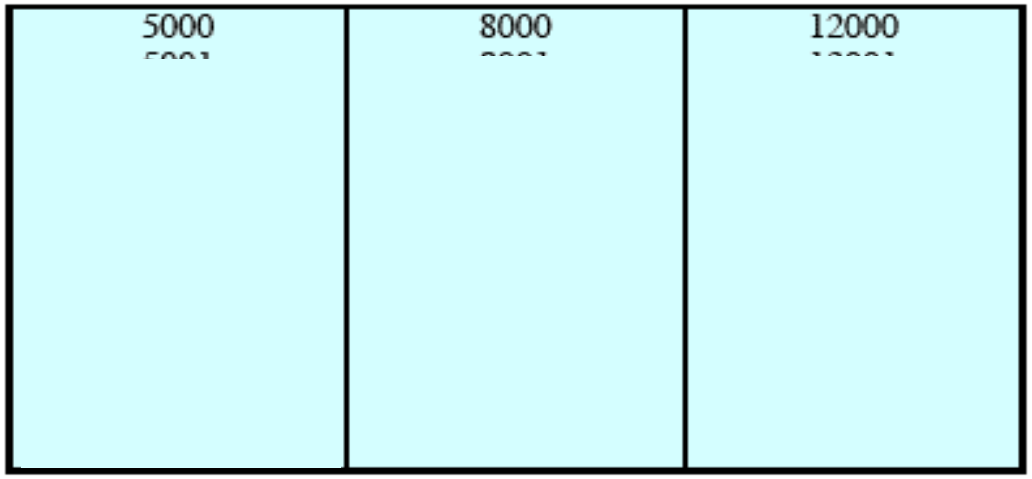
Process Execution



Consider three processes being executed
All are in memory (plus the dispatcher)
Lets ignore virtual memory for this.

Trace from the *processes* point of view

Each process runs to completion

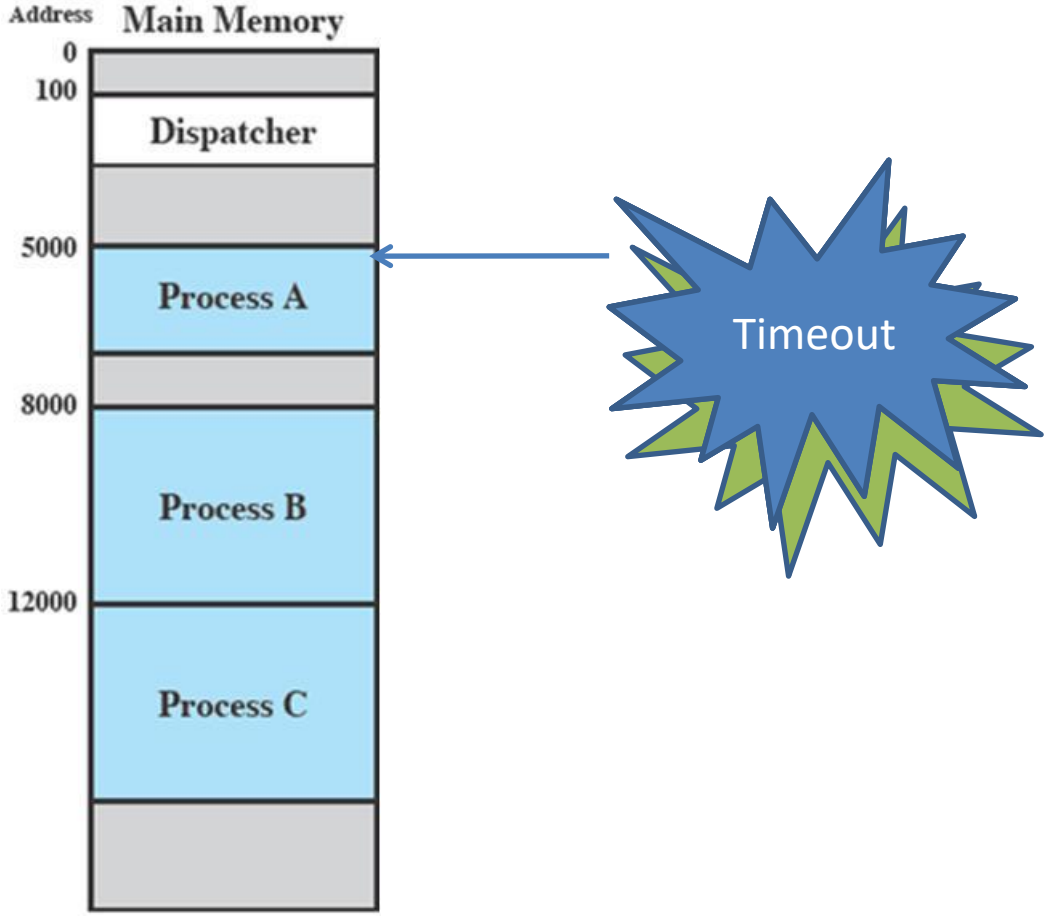


(a) Trace of Process A (b) Trace of Process B (c) Trace of Process C

5000 = Starting address of program of Process A
8000 = Starting address of program of Process B
12000 = Starting address of program of Process C

Figure 3.3 Traces of Processes of Figure 3.2

Trace from Processors point of view



Timeout

1	5000
2	5001
3	5002
4	5003
5	5004
6	5005
	Timeout
7	100
8	101
9	102
10	103
11	104
12	105
13	8000
14	8001
15	8002
16	8003
	I/O Request
17	100
18	101
19	102
20	103
21	104
22	105
23	12000
24	12001
25	12002
26	12003

27	12004
28	12005
	Timeout
29	100
30	101
31	102
32	103
33	104
34	105
35	5006
36	5007
37	5008
38	5009
39	5010
40	5011
	Timeout
41	100
42	101
43	102
44	103
45	104
46	105
47	12006
48	12007
49	12008
50	12009
51	12010
52	12011
	Timeout

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;
first and third columns count instruction cycles;
second and fourth columns show address of instruction being executed

Figure 3.4 Combined Trace of Processes of Figure 3.2

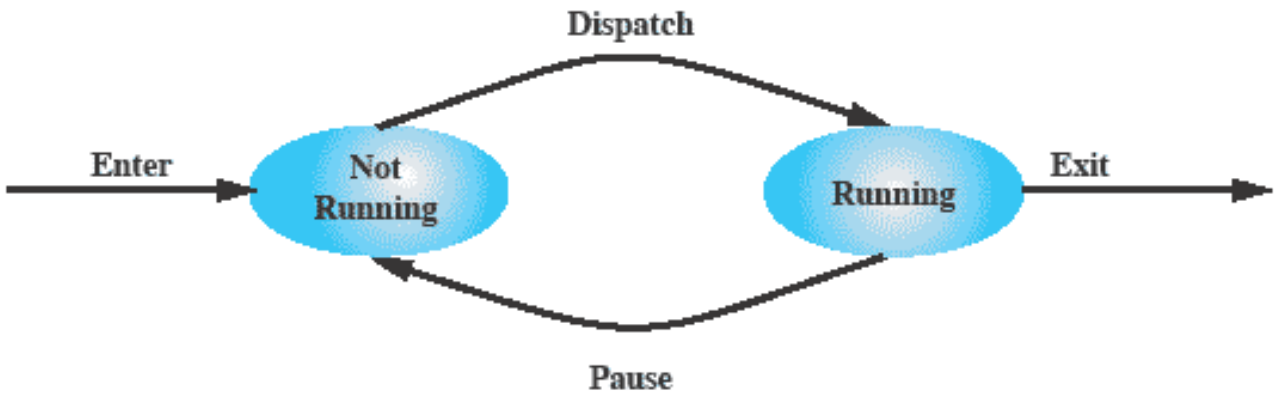
Roadmap

- How are processes represented and controlled by the OS.
  - ***Process states*** which characterize the behaviour of processes.
  - ***Data structures*** used to manage processes.
  - Ways in which the OS uses these data structures to control process execution.
  - Discuss process management in UNIX SVR4.

Two-State Process Model

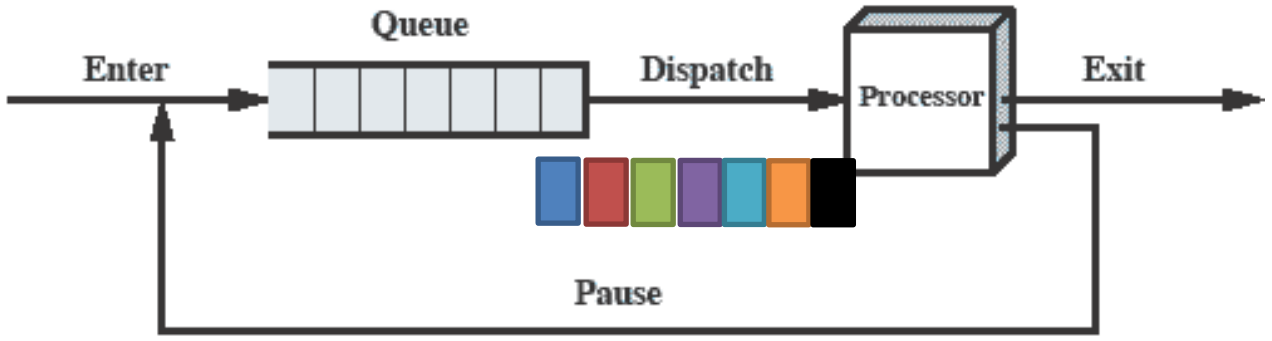
Process may be in one of two states

- Running
- Not-running



(a) State transition diagram

Queuing Diagram



(b) Queuing diagram

Etc … processes moved by the dispatcher of the OS to the CPU then back to the queue until the task is competed

Process Birth and Death

Creation	Termination
New batch job	Normal Completion
Interactive Login	Memory unavailable
Created by OS to provide a service	Protection error
Spawned by existing process	Operator or OS Intervention

See tables 3.1 and 3.2 in book for more

Process Creation

The OS builds a data structure to manage the process

Traditionally, the OS created all processes

- But it can be useful to let a running process create another

This action is called ***process spawning***

- ***Parent Process*** is the original, creating, process
- ***Child Process*** is the new process

Process Termination

There must be some way that a process can indicate completion.
This indication may be:

- A HALT instruction generating an interrupt alert to the OS.
- A user action (e.g. log off, quitting an application)
- A fault or error
- Parent process terminating

Five-State Process Model

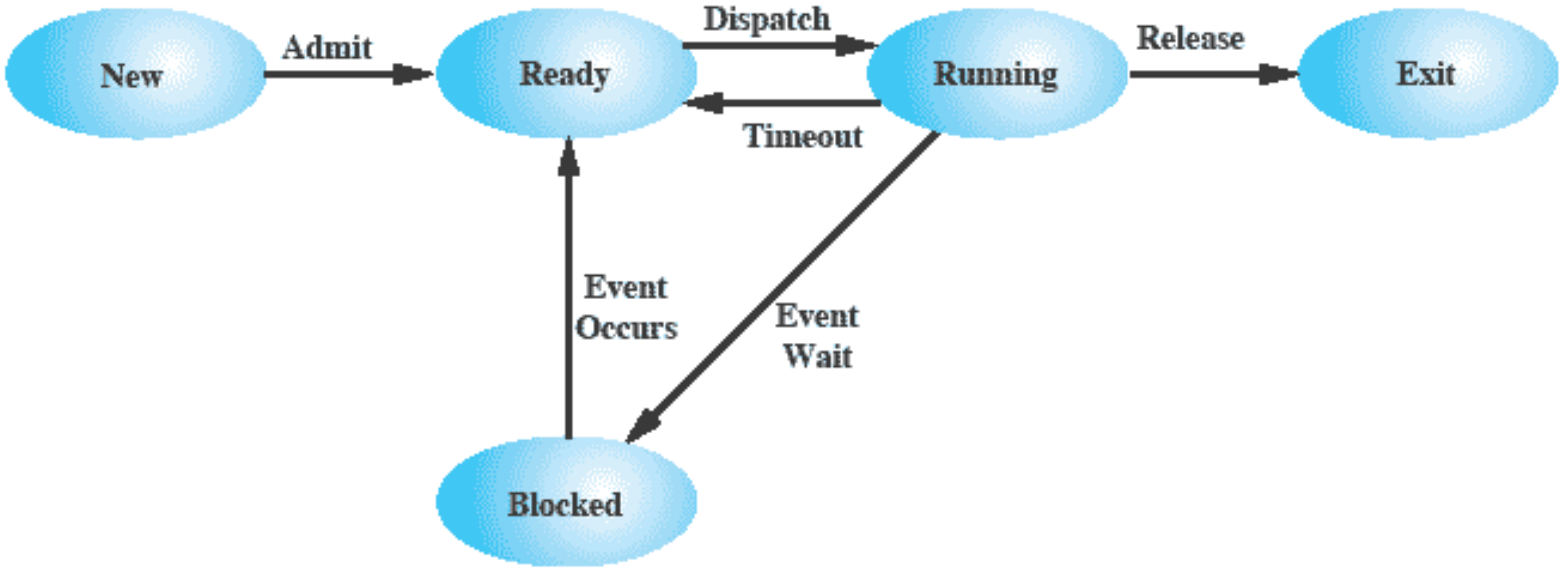
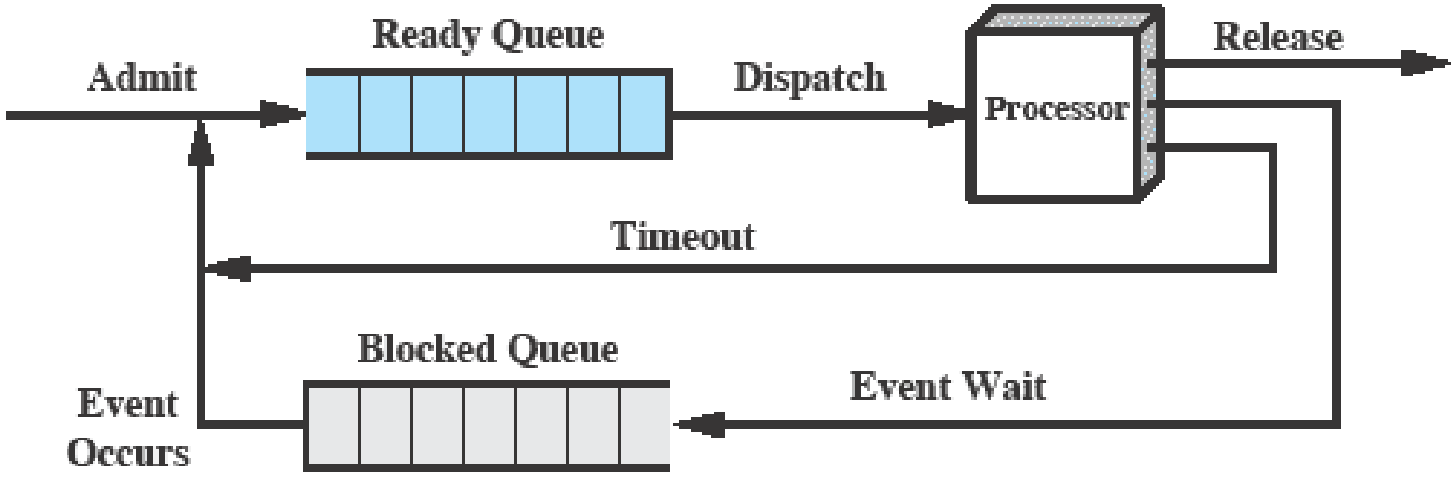


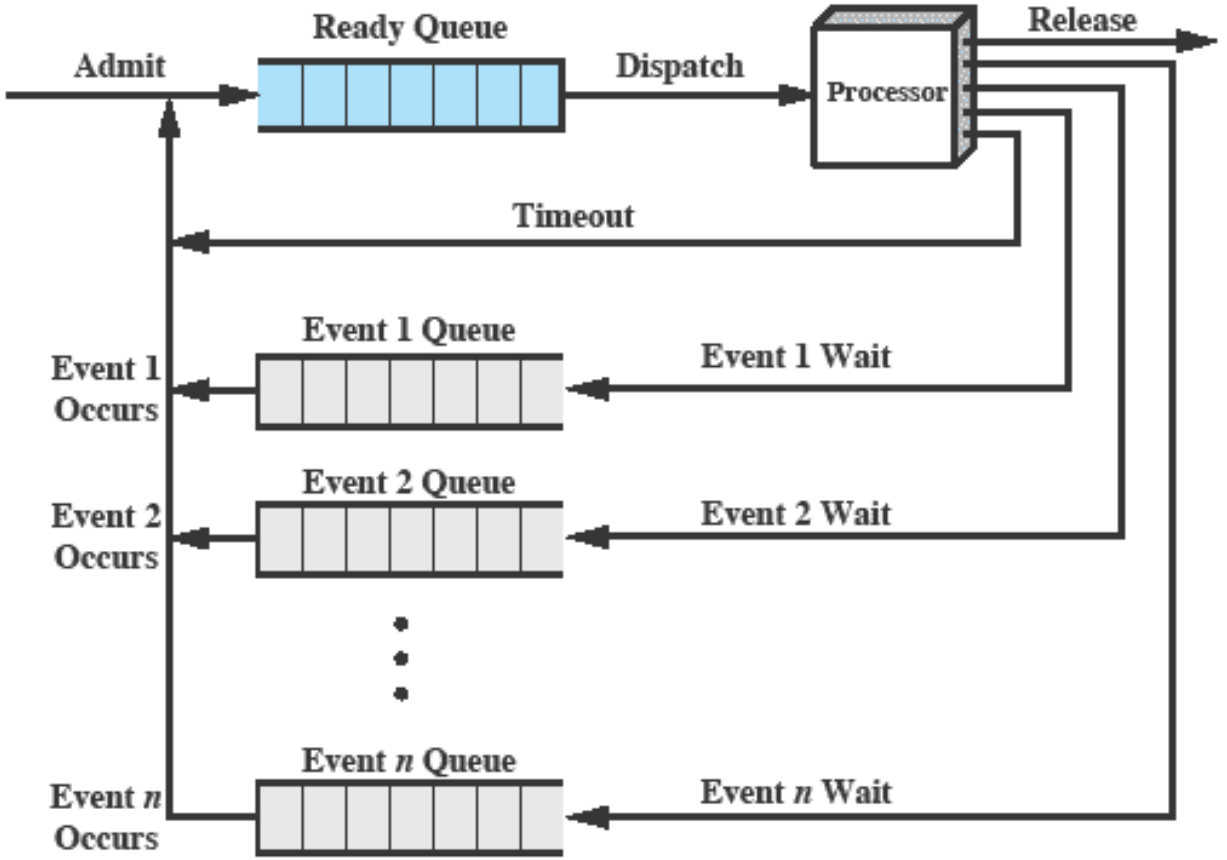
Figure 3.6 Five-State Process Model

Using Two Queues



(a) Single blocked queue

Multiple Blocked Queues

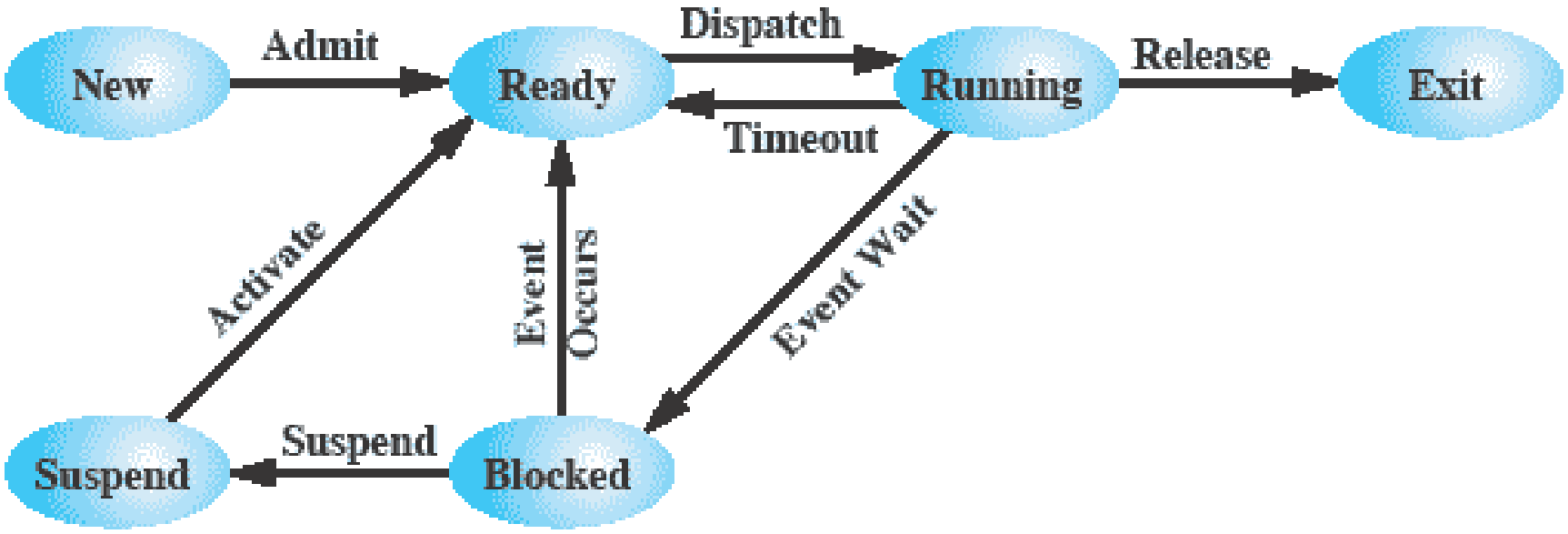


(b) Multiple blocked queues

Suspended Processes

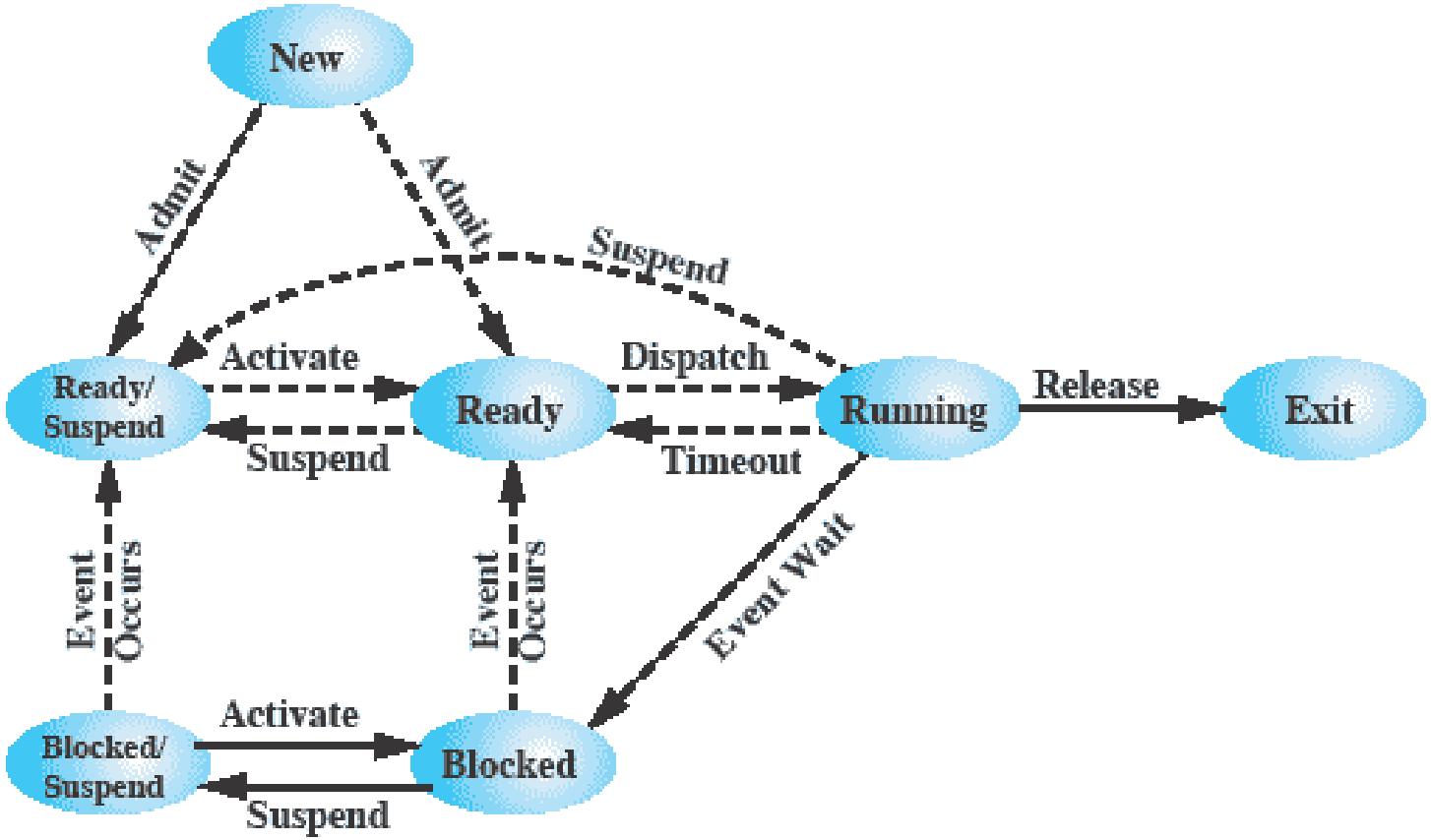
- Processor is faster than I/O so all processes could be waiting for I/O
  - Swap these processes to disk to free up more memory and use processor on more processes
- Blocked state becomes ***suspend*** state when swapped to disk
- Two new states
  - Blocked/Suspend
  - Ready/Suspend

One Suspend State



(a) With One Suspend State

Two Suspend States



(b) With Two Suspend States

Reason for Process Suspension

Reason	Comment
Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS Reason	OS suspects process of causing a problem.
Interactive User Request	e.g. debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time.
Parent Process Request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Table 3.3 Reasons for Process Suspension

Roadmap

- How are processes represented and controlled by the OS.
- ***Process states*** which characterize the behaviour of processes.
- ***Data structures*** used to manage processes.
- Ways in which the OS uses these data structures to control process execution.
- Discuss process management in UNIX SVR4.

Processes and Resources

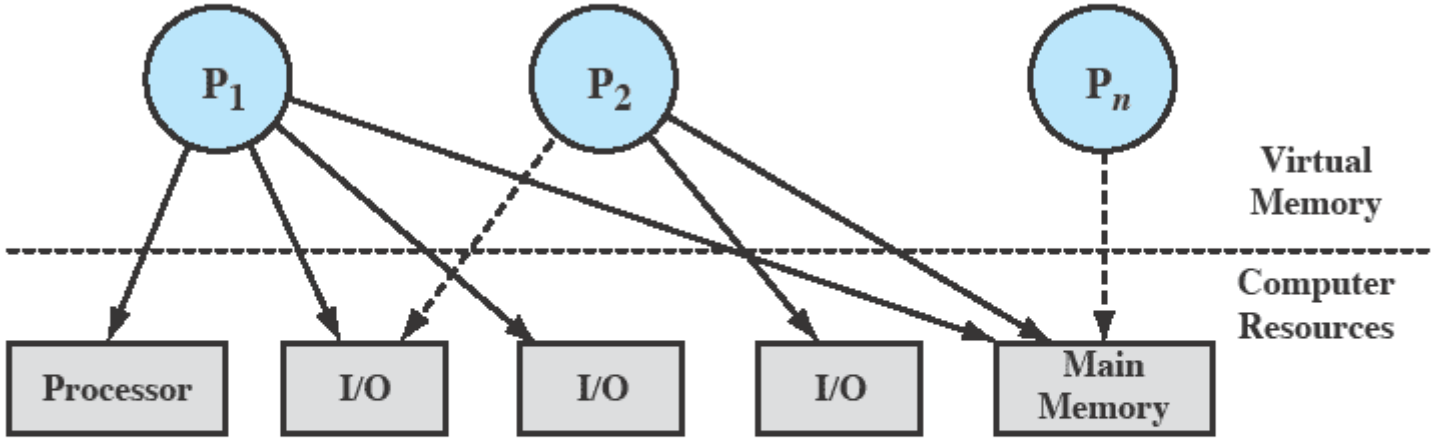


Figure 3.10 Processes and Resources (resource allocation at one snapshot in time)

Operating System Control Structures

For the OS is to manage processes and resources, it must have information about the current status of each process and resource. Tables are constructed for each entity the operating system manages

OS Control Tables

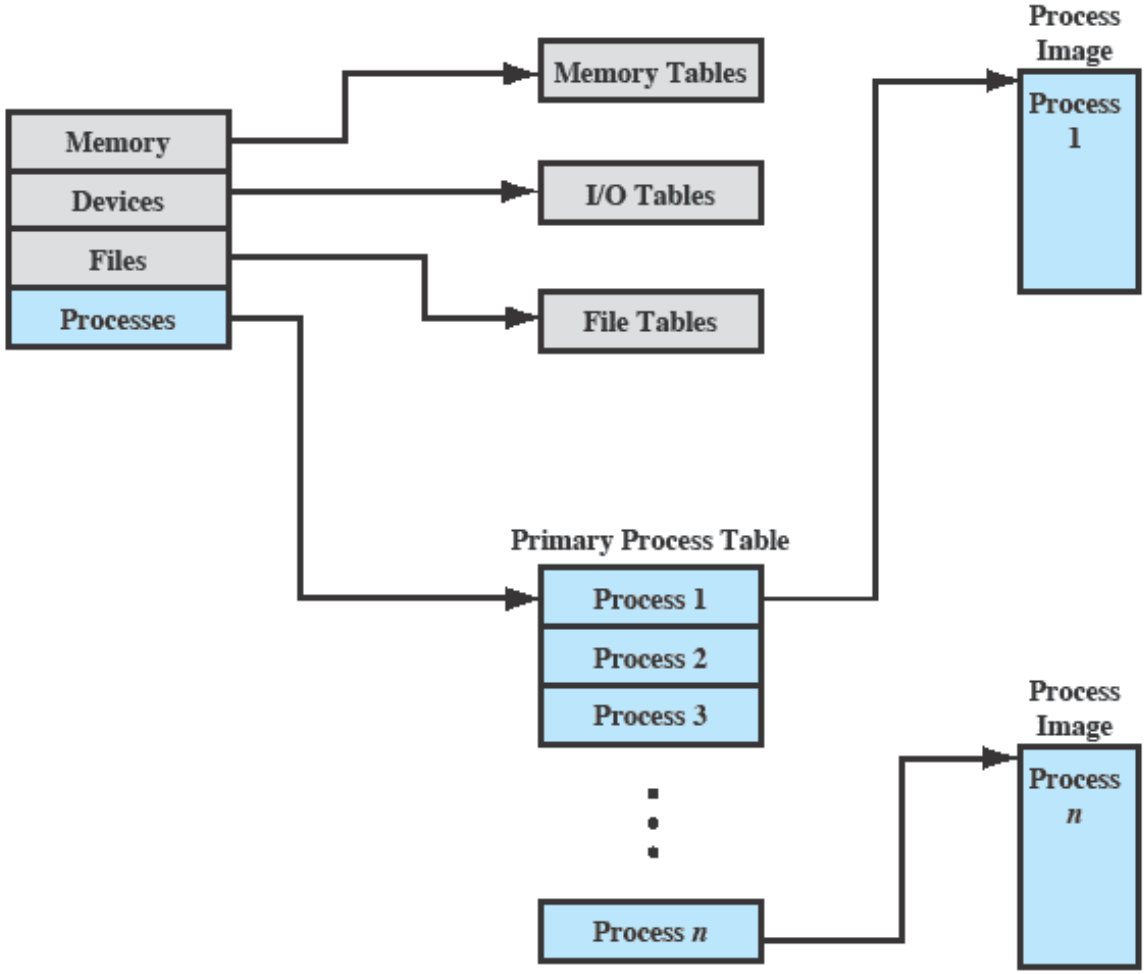


Figure 3.11 General Structure of Operating System Control Tables

Memory Tables

Memory tables are used to keep track of both main and secondary memory.

Must include this information:

- Allocation of main memory to processes
- Allocation of secondary memory to processes
- Protection attributes for access to shared memory regions
- Information needed to manage virtual memory

I/O Tables

Used by the OS to manage the I/O devices and channels of the computer.

The OS needs to know

- Whether the I/O device is available or assigned
- The status of I/O operation
- The location in main memory being used as the source or destination of the I/O transfer

File Tables

These tables provide information about:

- Existence of files
- Location on secondary memory
- Current Status
- other attributes.

Sometimes this information is maintained by a file management system

Process Tables

To manage processes the OS needs to know details of the processes

- Current state
- Process ID
- Location in memory
- etc

Process control block

- ***Process image*** is the collection of program. Data, stack, and attributes

Process Attributes

We can group the process control block information into three general categories:

- Process identification
- Processor state information
- Process control information

Process Identification

Each process is assigned a unique numeric identifier.
Many of the other tables controlled by the OS may use process identifiers to cross-reference process tables

Processor State Information

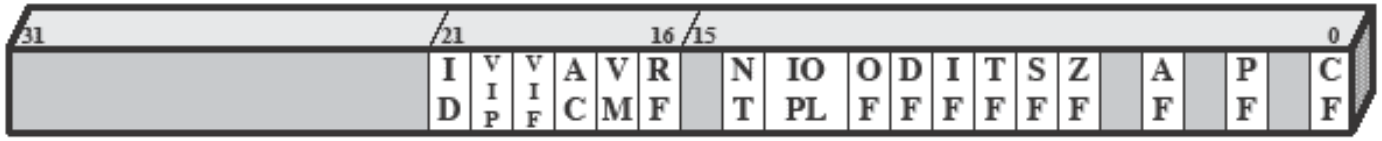
This consists of the contents of processor registers.

- User-visible registers
- Control and status registers
- Stack pointers

Program status word (PSW)

- contains status information
- Example: the EFLAGS register on Pentium processors

Pentium II
EFLAGS Register



ID = Identification flag
VIP = Virtual interrupt pending
VIF = Virtual interrupt flag
AC = Alignment check
VM = Virtual 8086 mode
RF = Resume flag
NT = Nested task flag
IOPL = I/O privilege level
OF = Overflow flag

DF = Direction flag
IF = Interrupt enable flag
TF = Trap flag
SF = Sign flag
ZF = Zero flag
AF = Auxiliary carry flag
PF = Parity flag
CF = Carry flag

Also see Table 3.6

Figure 3.12 Pentium II EFLAGS Register

Process Control Information

This is the additional information needed by the OS to control and coordinate the various active processes.

See table 3.5 for scope of information

Structure of Process Images in Virtual Memory

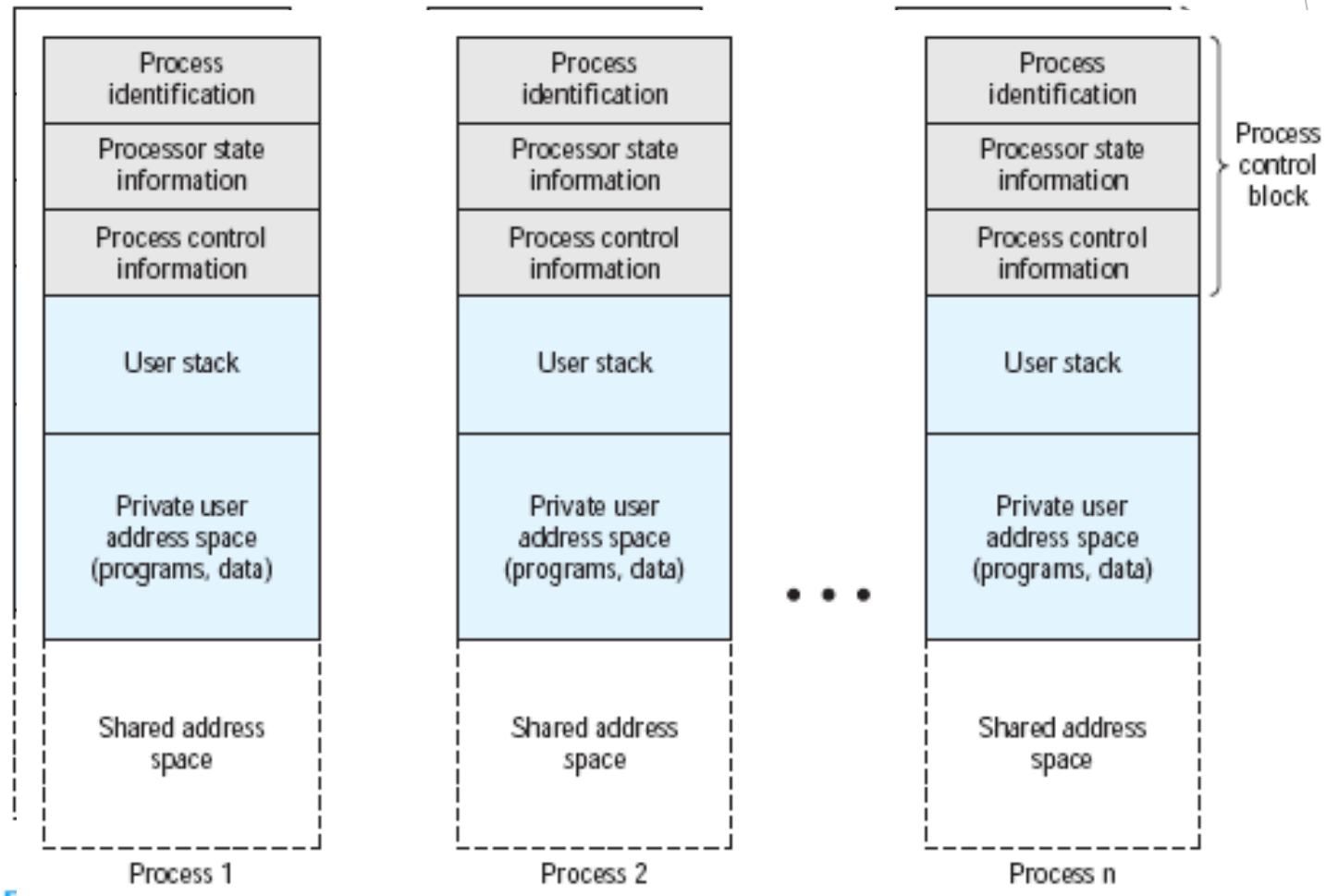


Figure 3.13 User Processes in Virtual Memory